IIITDM INTERNSHIP PROJECT

Multiclass UAV Detection and Localization

Final Project Report

PROJECT SCOPE

TinyML object detection | Knowledge distillation | INT8 deployment
Multiclass detection: Airplane | Bird | Drone | UAV

TinyissimoYOLO Repository Model Selection

Source. The supplied Git repository is the TinyissimoYOLO codebase associated with the ETH Zurich work *Ultra-Efficient On-Device Object Detection on AI-Integrated Smart Glasses with TinyissimoYOLO*. For this project, four TinyissimoYOLO teacher families were retained: **V1-Small, V1, V5-b and V8-b**.

Selected TinyissimoYOLO configurations

Model / version	Repository configuration	Parameters in supplied ZIP*	Role in this project
TinyissimoYOLO V1-Small	tinyissimo-v1-small.yaml	353,512	Compact fixed V1 architecture
TinyissimoYOLO V1	tinyissimo-v1.yaml	910,840	Larger fixed V1 architecture
TinyissimoYOLO V5-b	tinyissimo-v5.yaml, scale b	888,784	Selected V5 scale
TinyissimoYOLO V8-b	tinyissimo-v8.yaml, scale b	839,392	Selected V8 scale

Parameter counts are instantiated directly from the supplied repository YAMLs in their original **nc=20 configuration; later project-specific 4-class training can change detection-head parameter counts.*

V5 and V8 compound-scale options in the repository

Scale	V5 scale [depth, width, max ch.]	V5 parameters	V8 scale [depth, width, max ch.]	V8 parameters
b	[0.33, 0.15, 1024]	888,784	[0.30, 0.18, 1024]	839,392
n	[0.33, 0.25, 1024]	2,202,896	[0.33, 0.25, 1024]	1,451,904
s	[0.33, 0.50, 1024]	8,777,648	[0.33, 0.50, 1024]	5,689,264
m	[0.67, 0.75, 1024]	24,824,464	[0.67, 0.75, 768]	13,554,832
l	[1.00, 1.00, 1024]	53,169,392	[1.00, 1.00, 512]	23,452,464
x	[1.33, 1.25, 1024]	97,206,992	[1.00, 1.25, 512]	36,632,592

Why the b-scale was selected for V5 and V8

The repository provides **b, n, s, m, l and x** compound scales for both V5 and V8. In this TinyissimoYOLO fork, the **b** configuration uses the smallest width multiplier (0.15 for V5 and 0.18 for V8), so it has the **lowest parameter count** among the listed V5/V8 variants. Therefore V5-b and V8-b were chosen to keep teacher architecture and model size manageable for the resource-constrained project, while the larger n/s/m/l/x alternatives introduce substantially more parameters and memory/computation.

Multiclass UAV Dataset Preparation and Reduction

Dataset source. The project dataset was obtained from **Roboflow** as a multiclass aerial-object detection dataset. The four detection classes used throughout the project are **Airplane, Bird, Drone and UAV**.

Dataset evolution

Stage	Image count / split	Details
Original dataset	30,720 total	Initial multiclass aerial-object dataset before project preprocessing.
Initial split	80% / 10% / 10%	Train = 24,576; Test = 3,072; Validation = 3,072.
Preprocessing	Same detection data	RGB images were converted to **grayscale** as part of preprocessing.
After preprocessing + augmentation	48,554 images	Augmentation expanded the working image pool; the supplied notes do not specify individual augmentation operators.
After training-set reduction	Train = 19,997	Duplicate and visually redundant samples were removed from the training set.
Final validation set	3,072	Kept unchanged for evaluation.
Final test set	3,072	Kept unchanged for final testing.
Final dataset	**26,141 total**	19,997 train + 3,072 validation + 3,072 test.

Training-set duplicate and similarity reduction

Technique	Purpose in the reduction pipeline
SHA-256 exact hashing	Identifies byte-identical duplicate images using a secure content hash; exact repeats can then be removed.
Perceptual hashing (pHash)	Represents visual content compactly so near-duplicate images can be detected even when file bytes differ.
BK-tree (Burkhard-Keller tree)	Indexes perceptual hashes efficiently and finds visually similar images without exhaustive all-pairs comparison.
Hamming-distance threshold = 5	Images whose perceptual hashes fall within the selected distance are treated as highly similar candidates for reduction.

Why the training dataset was reduced

Reason 1 - remove redundant learning and reduce training time. After augmentation, some samples can be exact or visually near-identical. Training repeatedly on these redundant images increases epoch time and compute without providing equivalent new information.

Reason 2 - improve effective diversity and reduce overfitting to repeated patterns. A training set dominated by highly similar samples can bias optimization toward repeated scenes/objects. Keeping more unique examples improves the information content of the training set and makes the comparison of teacher models more meaningful, while the validation and test sets remain fixed for fair evaluation.

Final dataset used for the project: 19,997 training images, 3,072 validation images and 3,072 test images, giving **26,141 images** in total across four classes.

Arduino Nano 33 BLE

The Arduino Nano 33 BLE is a compact 3.3 V development board based on the Nordic nRF52840 microcontroller. It provides a 32-bit ARM Cortex-M4 processor, useful memory capacity, Bluetooth connectivity and standard communication interfaces, making it suitable for embedded systems, wireless sensing and TinyML applications.

Main Specifications

Feature	Details
Microcontroller	Nordic nRF52840, 32-bit ARM Cortex-M4, 64 MHz
Memory	1 MB Flash and 256 KB SRAM
Operating Voltage	3.3 V logic
Wireless	Bluetooth 5 through the u-blox NINA-B306 module
Communication	USB, I2C, SPI and UART
Digital I/O	14 digital I/O pins with PWM support
Analog Inputs	8 analog input pins

Use in TinyML

Its nRF52840 processor and 256 KB SRAM allow compact machine-learning models to run directly on the board using frameworks such as TensorFlow Lite Micro. For image-processing applications, small models, pruning and INT8 quantization are useful for reducing Flash, SRAM and computation requirements.

Important Note

The board uses 3.3 V logic. Applying 5 V directly to its I/O pins can damage the board.

Verified Multiclass UAV Detection Model Comparison

Test-set metrics verified against the uploaded JSON reports and notebook outputs. Values are rounded to 3 decimals for the comparison table.

Model	Input	Parameters	GFLOPs	INT8 Size (KB)	mAP50 (%)	mAP50-95 (%)	Precision (%)	Recall (%)	F1 (%)	Latency (ms)	FPS
V1	128x128	958,268	0.455271	1032.313	66.829	44.393	75.593	68.077	71.638	9.162	109.147
V1	256x256	958,268	1.821086	1033.279	78.625	54.754	84.164	76.350	80.067	18.084	55.298
V1-small	128x128	352,448	0.128028	417.190	63.120	40.818	74.806	61.993	67.799	10.846	92.200
V1-small	256x256	352,448	0.512113	418.165	75.142	36.851	83.002	70.168	76.047	12.065	82.882
V5-b	128x128	883,488	0.108089	1152.288	75.357	48.360	78.696	71.579	74.969	18.705	53.461
V5-b	256x256	883,488	0.432355	1172.875	88.857	46.589	91.074	81.590	86.072	21.015	47.585
V8-b	128x128	827,848	0.088171	1063.691	77.912	50.960	79.287	78.195	78.737	26.864	37.225
V8-b	256x256	827,848	0.352684	1063.687	88.778	46.039	90.523	83.969	87.122	18.209	54.917
YOLO11n	640x640	2,590,620	6.443827	3300.000	95.805	68.312	95.251	92.453	93.831	13.255	75.443

Verification notes. All parameters, GFLOPs, test mAP50, mAP50-95, precision, recall, F1, latency and FPS were checked against the uploaded reports/notebook outputs. Corrections versus the handwritten sheet include V1-small 128 mAP50-95 = **40.818%**, V1-small 256 mAP50 = **75.142%**, V5-b 128 recall = **71.579%**, and V8-b 128 precision = **79.287%**.
For Tinyissimo rows, the INT8-size values are retained from the comparison data. YOLO11n INT8 size is shown as **3300 KB** as requested.
Metric convention: latency is the reported total per-image latency and FPS is the corresponding end-to-end/reported FPS. YOLO11n values come from the OpenVINO INT8 final summary; Tinyissimo values come from their test-split reports.

UAV Multiclass Teacher-Student Knowledge Distillation Pipeline

Project-specific workflow: TinyissimoYOLO teachers -> 15-block Custom CNN TinyML student -> INT8 deployment

1 UAV MULTICLASS RGB DATASET
4 classes: airplane | bird | drone | uav - YOLO-format bounding-box annotations

A. TEACHER BRANCH - SOFT TARGETS

2A RGB UAV images
Resize to 256 x 256 x 3

3A Frozen teacher (one KD run at a time):
v1 | v1_small | v5b | v8b
Teacher input: 256 x 256

teacher remains frozen

4A Pre-NMS teacher predictions
soft class logits / scores + box information

5A Soft-target / KD encoder
aligns teacher knowledge to student target format for distillation during training

B. HARD-LABEL BRANCH - GT

2B Original YOLO ground-truth labels
class_id, x_center, y_center, width, height

3B Hard-target encoder
assigns objects to 8 x 8 grid cells

4B 8 x 8 x 9 hard target grid
[x, y, w, h, objectness, 4 class targets]

C. STUDENT BRANCH - CUSTOM CNN

2C RGB student input
Resize to 128 x 128 x 3

3C 15-BLOCK CUSTOM CNN STUDENT
Compact depthwise-separable CNN
Filter / channel flow:
16 -> 16 -> 32 -> 32 -> 64 -> 64 -> 96 -> 96 ->
112 -> 112 -> 128 -> 104 -> 80 -> 48 -> 9
Conv + BN + ReLU / Depthwise + Pointwise blocks
Single detection head

4C 8 x 8 x 9 prediction tensor
[x, y, w, h, objectness + 4 class logits]

D. LOSS COMPUTATION + STUDENT UPDATE

5 Hard-label detection loss + Knowledge-distillation loss
GT terms: localization (CIoU), objectness, multiclass classification
KD terms: teacher soft class information and teacher box guidance
Backpropagation updates STUDENT ONLY - teachers remain frozen

E COMPRESSION AND DEPLOYMENT - STUDENT ONLY

6 10% magnitude pruning
Compact student sparsification

7 Recovery fine-tuning
Restore post-pruning accuracy

8 Full INT8 quantization
Representative-data calibration

9 INT8 TFLite model
128 x 128 x 3 -> 8 x 8 x 9

10 Arduino C header
TFLite Micro deployment files

11 Nano 33 BLE Sense
Final multiclass UAV detection

FINAL OUTPUT: bounding box + class label (airplane / bird / drone / uav) + confidence score
Post-processing: confidence filtering + class-aware NMS

IMPORTANT: v1, v1_small, v5b and v8b are used as separate frozen teachers in separate KD experiments (not an ensemble).
Only the pruned INT8 Custom CNN student is deployed on Arduino Nano 33 BLE Sense.

Custom CNN Architecture Report

15 major blocks | Input 128x128x3 | Output 8x8x9 | 105 Keras layers including input | 34 Conv/Depthwise layers

Major blocks	Keras layers	Parameters	Approx. GFLOPs	Filter flow	Final output
15	105 (incl. input)	133,305	0.033337344	16 -> 16 -> 32 -> 64 -> 96 -> 112 -> 128 -> 104 -> 80 -> 48 -> 9	8x8x9

Table 1. Complete 15-block architecture

Block	Keras layers	Channels	Kernel / stride	Operation	Output feature map
1	3	16	3x3, s=2	Standard Conv + BN + ReLU	64x64x16
2	6	16	3x3 DW + 1x1 PW, s=1	Depthwise-separable Conv	64x64x16
3	6	32	3x3 DW + 1x1 PW, s=2	Downsampling depthwise-separable Conv	32x32x32
4	13	32	2 x (3x3 DW + 1x1 PW), s=1	Residual depthwise-separable block + Add	32x32x32
5	6	64	3x3 DW + 1x1 PW, s=2	Downsampling depthwise-separable Conv	16x16x64
6	13	64	2 x (3x3 DW + 1x1 PW), s=1	Residual depthwise-separable block + Add	16x16x64
7	6	96	3x3 DW + 1x1 PW, s=2	Downsampling depthwise-separable Conv	8x8x96
8	13	96	2 x (3x3 DW + 1x1 PW), s=1	Residual depthwise-separable block + Add	8x8x96
9	6	112	3x3 DW + 1x1 PW, s=1	Depthwise-separable Conv	8x8x112
10	13	112	2 x (3x3 DW + 1x1 PW), s=1	Residual depthwise-separable block + Add	8x8x112
11	6	128	3x3 DW + 1x1 PW, s=1	Depthwise-separable Conv	8x8x128
12	6	104	3x3 DW + 1x1 PW, s=1	Depthwise-separable Conv / compression	8x8x104
13	3	80	1x1, s=1	Pointwise Conv head + BN + ReLU	8x8x80
14	3	48	1x1, s=1	Pointwise Conv head + BN + ReLU	8x8x48
15	1	9	1x1, s=1	Final raw detector Conv	8x8x9

Feature-map reduction
128x128 input -> 64x64 (Blocks 1-2) -> 32x32 (Blocks 3-4) -> 16x16 (Blocks 5-6) -> 8x8 (Blocks 7-15).

3x3 depthwise + 1x1 pointwise
3x3 depthwise layers learn spatial patterns such as edges, shapes and object boundaries; 1x1 pointwise layers mix and compress channel information.

Detector output
Each 8x8 cell produces 9 values: tx, ty, w, h, objectness and four class logits for airplane, bird, drone and UAV.

Custom CNN Final INT8 TFLite Software Results

Final 4-class detection and inference results from the trained, pruned and fully quantized Custom CNN

Table 2. Model configuration and complexity

Item	Value	Unit / description
Model	Custom CNN Prune10 15Blocks 128 Multiclass	-
Task	4-class object detection with bounding-box localization	-
Classes	Airplane, Bird, Drone, UAV	4 classes
Input size	128x128x3	RGB
Output size	8x8x9	Raw detector tensor
Major blocks	15	Blocks
Keras layers	105	Including input layer
Conv / Depthwise layers	34	Convolutional processing layers
Parameters	133,305	Parameters
Approx. computation	0.033337344	GFLOPs
INT8 model size	229.414	KiB
Final pruning sparsity	~10.001	%

Table 3. Final INT8 detection and inference performance

Metric	Final value	Unit
mAP50	**60.970**	%
mAP50:95	**32.640**	%
Precision	**73.210**	%
Recall	**70.820**	%
F1-score	**71.990**	%
Latency	**1.384742**	ms / image
FPS	**722.156185**	frames / second

Architecture interpretation
The 15-block network progressively reduces feature-map resolution from 128x128 to 8x8 while increasing channel depth. Depthwise-separable convolutions reduce computational cost, residual blocks preserve information, and 1x1 heads compress the learned features before the detector output.

Deployment interpretation
The final INT8 model occupies 229.414 KiB. The final software detection performance is 60.970% mAP50 with 73.210% precision, 70.820% recall and 71.990% F1. Local-CPU TFLite latency is 1.384742 ms, corresponding to 722.156185 FPS.

Source basis: uploaded Custom CNN notebook and handwritten architecture/result notes. Final INT8 software values are reported without threshold annotation, as requested.

Teacher-KD Software TFLite INT8 Comparison - 5 Models

Software comparison of the same CustomCNN128 student distilled from five teacher models.

Model	Teacher Input Size	Parameters	GFLOPs	INT8 Model Size (KiB)	mAP50 (%)	Precision (%)	Recall (%)	F1 (%)	Latency (ms)	FPS
V1	256x256	133,305	0.033337344	234.804	64.180	69.710	73.670	71.640	1.052	950.387
V1-Small	256x256	133,305	0.033337344	237.125	64.080	72.680	72.910	72.800	0.368	2711.085
V5-B	256x256	133,305	0.033337344	235.179	63.600	74.720	72.370	73.530	0.920	1085.927
V8-B	256x256	133,305	0.033337344	229.414	59.800	73.870	69.290	71.510	0.831	1202.629
YOLOV11	640x640	133,305	0.033337344	229.414	64.238	69.809	75.412	72.502	1.073	931.904

Notes: Values are displayed to three decimal places. Parameters (133,305) and approximate GFLOPs (0.033337344) are the same because all five experiments use the same CustomCNN128 student architecture. Teacher Input Size refers to the teacher network input used during knowledge distillation; the deployed student TFLite network uses 128x128x3 input.

INT8 TFLite Software vs Arduino Hardware Comparison - 6 Models

3072 test images | Software: local CPU TFLite INT8 | Hardware: Arduino Nano 33 BLE, TFLite Micro | Hardware latency = mean inference time

Model	Software TFLite INT8						Hardware TFLite Micro					
	mAP50 (%)	Precision (%)	Recall (%)	F1 (%)	Latency (ms)	FPS	mAP50 (%)	Precision (%)	Recall (%)	F1 (%)	Latency (ms)	FPS
Custom CNN	60.970	73.210	70.820	71.990	1.384	722.156	60.490	72.846	70.625	71.718	1703.106	0.587
V1	64.180	69.710	73.670	71.640	1.052	950.387	64.278	69.687	73.737	71.655	1702.647	0.587
V1 SMALL	64.080	72.680	72.910	72.800	0.368	2711.085	63.805	72.519	72.657	72.588	1702.655	0.587
V5B	63.600	74.720	72.370	73.530	0.920	1085.927	64.157	74.194	72.403	73.288	1696.370	0.589
V8B	59.800	73.870	69.290	71.510	0.831	1202.629	59.091	74.087	69.006	71.456	1696.785	0.589
YOLO11	64.238	69.809	75.412	72.502	1.073	931.904	64.582	70.162	75.349	72.663	1696.363	0.589

YOLO11 final hardware run (all 3072 images): FINAL TOTAL mAP50 = **68.104%**, FINAL TOTAL mAP50:95 = **36.624%**. For the deployment-threshold comparison used in the table: mAP50 @0.75 = **64.582%**, Precision = **70.162%**, Recall = **75.349%**, F1 = **72.663%**, mean latency = **1696.363 ms**, FPS = **0.589497**. The software mAP50 @0.75 reference is 64.238%, giving a hardware-software delta of +0.344 percentage points.

Literature Survey Papers for Multiclass UAV Project

17 published papers related to TinyissimoYOLO, YOLO11, multiclass UAV object detection, knowledge distillation and embedded deployment.

No.	Paper	Year	Model	Classes / Dataset	mAP50 / Reported Result	KD	Teacher-Student	Hardware Deployment	Quantization / Method
1	**TinyissimoYOLO: A Quantized, Low-Memory Footprint, TinyML Object Detection Network for Low Power Microcontrollers**	2023	TinyissimoYOLO (~422k params)	Up to 3 classes; Pascal VOC subset	Overall mAP 58.5% (not explicitly mAP50); up to 180 FPS on MAX78000	No	No	Yes - MAX78000, STM32H7A3, STM32L4R9, Apollo4b	QAT + 8-bit quantization; ultra-light MCU detector
2	**Flexible and Fully Quantized Lightweight TinyissimoYOLO for Ultra-Low-Power Edge Systems**	2024	Flexible TinyissimoYOLO variants	3, 10 and 20 classes; Pascal VOC	mAP 63.1% for TY:3-3-112; largest deployable 20-class variant mAP 56.4%	No	No	Yes - GAP9, MAX78000, STM32H7A3, STM32L4R9, Apollo4b	Fully integer / 8-bit QAT; architecture and class-count scaling
3	**Ultra-Efficient On-Device Object Detection on AI-Integrated Smart Glasses with TinyissimoYOLO**	2025	TinyissimoYOLO v1.3, v5, v8	Pascal VOC 20 classes; COCO capability up to 80	TY-v8-Big: 44% mAP50:95; 17 ms network inference; ~18 FPS end-to-end	No	No	Yes - GAP9 smart-glasses prototype	Quantized sub-million-parameter YOLO family; energy/latency co-design
4	**DSORT-MCU: Detecting Small Objects in Real-Time on Microcontroller Units**	2024	TinyissimoYOLO v1.3 + FOMO	CARPK: 1 class (car)	F1 = 0.91, MAE = 6.2; mAP50 not reported; 0.6-16.2 ms/inference	No	No	Yes - GAP9 MCU / NE16	INT8 deployment; adaptive image tiling for tiny objects
5	**Real-Time Small UAV Detection in Complex Airspace Using YOLOv11 with Residual Attention and High-Resolution Feature Enhancement**	2026	YOLOv11n + ResCBAM + P2 head	3 classes: drone, bird, airplane	mAP50 96.9%; mAP50:95 84.5%; P 96.8%; R 92.1%; 50.51 FPS	No	No	No physical edge board reported	Residual attention + high-resolution P2 small-object head
6	**A Novel Dataset and Detection Method for Unmanned Aerial Vehicles Using an Improved YOLOv9 Algorithm**	2024	DotD-YOLOv9-C	4 classes: UAV, bird, helicopter, airplane	Overall mAP 92.6% (not explicitly mAP50); UAV AP 97.8%, bird 87.1%, helicopter 94.2%, airplane 91.4%	No	No	No embedded-board deployment reported	Dot-distance sample assignment for small aerial targets

Literature Survey Papers for Multiclass UAV Project

No.	Paper	Year	Model	Classes / Dataset	mAP50 / Reported Result	KD	Teacher-Student	Hardware Deployment	Quantization / Method
7	**Lightweight UAV Object-Detection Method Based on Efficient Multidimensional Global Feature Adaptive Fusion and Knowledge Distillation**	2024	MGFAFNET-S teacher -> MGFAFNET-N student	VisDrone2021: 10 classes; SyluDrone: 1	AP50 35.6% on VisDrone2021; AP50 90.2% on SyluDrone; student 2.09M params	Yes - LCDD	Yes	Yes - NVIDIA Jetson Xavier NX (~27 FPS)	DBMA + DSPF + Localized Compensation Dual-Mask Distillation
8	**RDR-KD: A Knowledge Distillation Detection Framework for Drone Scenes**	2024	Teacher detector -> lightweight student	VisDrone2019: 10 classes	mAP50 not reported in accessible summary; student AP +3.3 points; APs +2.9 points	Yes - RDR-KD	Yes	No physical board reported	Regenerated, Decoupled and Responsive knowledge distillation
9	**A Novel Tensor Decomposition-Based Efficient Detector for Low-Altitude Aerial Objects With Knowledge Distillation Scheme**	2024	TDKD-Net	VisDrone 10; SeaDronesSee 5; UAVOD10 10; COCO 80	mAP50: 52.1% VisDrone; 87.8% SeaDronesSee; 50.8% UAVOD10; 63.6% COCO	Yes	Yes	No edge board; RTX 3090 Ti experiments	Tensor-train decomposition + response KD + attention + small-object head
10	**Accurate Recognition of UAVs on Multi-Scenario Perception with YOLOv9-CAG**	2025	YOLOv9-CAG	4 visible classes: static bird, flying bird, rotorcraft UAV, fixed-wing UAV	UAV mAP50 92.0% visible, 86.5% IR; visible all-class mAP50 77.2%	No	No	No embedded-board deployment reported	CAM + GAM attention + AKConv; visible/IR/audio perception
11	**Robust Real-Time Recognition of Drones and Birds in Complex Scenarios: A Multimodal Data Fusion Recognize Approach**	2025/2026	YOLOv10-PRD	4 visible classes: static bird, flying bird, rotary-wing UAV, fixed-wing UAV	mAP50 82.5% visible; 84.4% IR; 66.3% radar; ~41 FPS on low-power device	No	No	Yes - low-power device; board not specified	PSContextAggregation + RFAConv + DWConv; multimodal fusion
12	**A Lightweight and Accurate UAV Detection Method Based on YOLOv4**	2022	Lightweight YOLOv4 + MobileNet v1/v2/v3	4 UAV types; 20,365 images	Best mAP 93.52% (MobileNetv3), 62 FPS; MobileNetv1 93.14%, 82 FPS	No	No	No physical MCU/edge-board deployment reported	MobileNet backbone + depthwise-separable convolutions

Literature Survey Papers for Multiclass UAV Project

No.	Paper	Year	Model	Classes / Dataset	mAP50 / Reported Result	KD	Teacher-Student	Hardware Deployment	Quantization / Method
13	**CF-YOLO for Small Target Detection in Drone Imagery Based on YOLOv11 Algorithm**	2025	CF-YOLO based on YOLOv11n	VisDrone2019-DET: 10 classes	mAP50 44.9%; mAP50:95 27.5%; P 52.8%; R 43.4%	No	No	No physical edge board reported	CS-FPN + FRM + Sandwich Fusion + RFAConv + P2 head
14	**Enhanced YOLO11 for Lightweight and Accurate Drone-Based Maritime Search and Rescue Object Detection**	2025	Optimized YOLO11-s	SeaDronesSee: 6 classes	mAP50:95 34.2%; 10.7M params; 45.5 GFLOPs; 3.3 ms/image	No	No	No physical edge board reported	Shallow feature integration + SPD downsampling + CARAFE upsampling
15	**TOE-YOLO: Accurate and Efficient Detection of Tiny Objects in UAV Imagery**	2025	TOE-YOLO based on YOLO11n	VisDrone2019: 10 classes; UAV-ROD/CARPK/UAVDT	VisDrone mAP50 33.8%; mAP50:95 19.7%; P 45.0%; R 33.7%	No	No	Yes - Raspberry Pi 5	Adaptive rotated conv + GhostConv + attention fusion
16	**Enhanced YOLOv11n for Small Object Detection in UAV Imagery: Higher Accuracy with Fewer Parameters**	2026	Improved YOLOv11n	VisDrone2019: 10; TinyPerson: 2; HazyDet	VisDrone mAP50 31.1%; mAP50:95 17.2%; P 42.7%; 2.36M params	No	No	Yes - Huawei Atlas 200 DK; 80.2 FPS	MSEAF backbone + lightweight fusion/head + P2 small-object head
17	**An Enhanced Algorithm Integrating YOLOv11 and ByteTrack for Small-Object Detection and Tracking in Low-Altitude Remote Sensing Imagery**	2026	TCYOLO-SofByteTrack	VisDrone2019: 10 classes + highway dataset	mAP50 40.7%; mAP50:95 24.7%; P 50.0%; R 39.2%; TensorRT FP16 mAP50 41.7%	No	No	Yes - NVIDIA Jetson Orin NX; 33.02 FPS	YOLOv11 + triplet attention + cross-scale fusion + ByteTrack; TensorRT FP16

Direct Links - 17 Published Papers

No.	Paper	Direct DOI Link
1	TinyissimoYOLO: A Quantized, Low-Memory Footprint, TinyML Object Detection Network for Low Power Microcontrollers	https://doi.org/10.1109/AICAS57966.2023.10168657
2	Flexible and Fully Quantized Lightweight TinyissimoYOLO for Ultra-Low-Power Edge Systems	https://doi.org/10.1109/ACCESS.2024.3404878
3	Ultra-Efficient On-Device Object Detection on AI-Integrated Smart Glasses with TinyissimoYOLO	https://doi.org/10.1007/978-3-031-91989-3_17
4	DSORT-MCU: Detecting Small Objects in Real-Time on Microcontroller Units	https://doi.org/10.1109/JSEN.2024.3425904
5	Real-Time Small UAV Detection in Complex Airspace Using YOLOv11 with Residual Attention and High-Resolution Feature Enhancement	https://doi.org/10.3390/jimaging12030140
6	A Novel Dataset and Detection Method for Unmanned Aerial Vehicles Using an Improved YOLOv9 Algorithm	https://doi.org/10.3390/s24237512
7	Lightweight UAV Object-Detection Method Based on Efficient Multidimensional Global Feature Adaptive Fusion and Knowledge Distillation	https://doi.org/10.3390/electronics13081558
8	RDR-KD: A Knowledge Distillation Detection Framework for Drone Scenes	https://doi.org/10.1109/LGRS.2024.3398140
9	A Novel Tensor Decomposition-Based Efficient Detector for Low-Altitude Aerial Objects With Knowledge Distillation Scheme	https://doi.org/10.1109/JAS.2023.124029
10	Accurate Recognition of UAVs on Multi-Scenario Perception with YOLOv9-CAG	https://doi.org/10.1038/s41598-025-12670-8
11	Robust Real-Time Recognition of Drones and Birds in Complex Scenarios: A Multimodal Data Fusion Recognize Approach	https://doi.org/10.1007/s40747-025-02117-y
12	A Lightweight and Accurate UAV Detection Method Based on YOLOv4	https://doi.org/10.3390/s22186874
13	CF-YOLO for Small Target Detection in Drone Imagery Based on YOLOv11 Algorithm	https://doi.org/10.1038/s41598-025-99634-0
14	Enhanced YOLO11 for Lightweight and Accurate Drone-Based Maritime Search and Rescue Object Detection	https://doi.org/10.1371/journal.pone.0321920
15	TOE-YOLO: Accurate and Efficient Detection of Tiny Objects in UAV Imagery	https://doi.org/10.1007/s11554-025-01770-3
16	Enhanced YOLOv11n for Small Object Detection in UAV Imagery: Higher Accuracy with Fewer Parameters	https://doi.org/10.1038/s41598-026-35301-2
17	An Enhanced Algorithm Integrating YOLOv11 and ByteTrack for Small-Object Detection and Tracking in Low-Altitude Remote Sensing Imagery	https://doi.org/10.3390/rs18101547

How These 17 References Map to the Project

Literature Need	Best Papers	What They Support
TinyML / TinyissimoYOLO foundation	**1, 2, 3, 4**	Quantized low-memory YOLO, MCU deployment, class-count / accuracy trade-offs, GAP9, MAX78000 and STM32-class platforms.
Closest UAV / multiclass detection setup	**5, 6, 10, 11**	Drone / UAV versus bird, airplane and fixed-wing classes; attention modules, small-target handling and multi-sensor robustness.
Knowledge distillation foundation	**7, 8, 9**	Teacher-student transfer, aerial small-object KD, response / logit distillation and compression-aware detector design.
YOLO11 small-object / multiclass image processing	**13, 14, 15, 16, 17**	YOLO11 feature fusion, high-resolution heads, lightweight architecture, small-target detection, multiclass aerial imagery and edge deployment.
Real hardware / edge deployment	**1-4, 7, 11, 15, 16, 17**	MCUs, Jetson Xavier NX, Raspberry Pi 5, Huawei Atlas 200 DK and Jetson Orin NX; quantized or FP16 deployment where reported.

Recommended citation use: compare methods, class setup, KD/compression, quantization and hardware platform. Do not claim that one paper's mAP is directly better or worse than another paper or your own project unless the same dataset, test split, IoU convention and evaluation protocol are used.